Université de Montpellier
Faculté des Sciences

HAI913I – Évolution et restructuration des logiciels

TP 1 – Analyse statique

Enseignant responsable : Bachar Rima
Année universitaire : 2026–2027

Étudiant : Lei Huang
Numéro étudiant : 22025417
Parcours : GL



Date : 11 octobre 2026
Dépôt Git :
https://github.com/LeiHuang25/HAI913I-evolution-et-restructuration-des-logiciels-TP1

Table des matières

1 Objectifs et périmètre **1**
1.1 Objectifs . . . 1
1.2 Périmètre fonctionnel . . . 1
1.3 Limites du périmètre . . . 1

2 Conception générale **1**
2.1 Chaîne d'analyse . . . 1
2.2 Architecture du projet . . . 2
2.3 Représentation du modèle de faits . . . 2

3 Réponses étape par étape **2**
3.1 A0 – Environnement et projet de validation . . . 2
3.1.1 Interprétation et stratégie . . . 2
3.1.2 Implémentation . . . 3
3.1.3 Résultats et validation . . . 3
3.2 A1 – Parcourir un AST . . . 3
3.2.1 Interprétation et stratégie . . . 3
3.2.2 Implémentation . . . 3
3.2.3 Résultats et validation . . . 3
3.3 A2 – Extraire la structure du projet . . . 4
3.3.1 Interprétation et stratégie . . . 4
3.3.2 Implémentation . . . 4
3.3.3 Résultats, validation et difficultés . . . 5
3.4 A3 – Extraire les appels et le type statique du receveur . . . 5
3.4.1 Interprétation et stratégie . . . 5
3.4.2 Implémentation . . . 6
3.4.3 Résultats, validation et difficultés . . . 6
3.5 B1 – Modèle de faits indépendant de JDT . . . 6
3.5.1 Choix de représentation . . . 6
3.5.2 Indépendance vis-à-vis de JDT et validation . . . 6
3.6 B2 – Calcul des métriques . . . 7
3.6.1 Définitions et conventions . . . 7
3.6.2 Résultats . . . 7
3.7 B3 – Graphe d'appel . . . 7
3.7.1 Construction du graphe . . . 7
3.7.2 Résultats et validation . . . 8
3.8 B4 – Outil utilisable et reproductible . . . 8
3.8.1 Interface en ligne de commande . . . 8
3.8.2 Gestion des erreurs et reproductibilité . . . 8
3.9 C1 – Extraction avec Spoon et comparaison . . . 9
3.9.1 Stratégie . . . 9
3.9.2 Différences de comptage . . . 9
3.9.3 Robustesse en mode `noClasspath` . . . 10
3.10 C2 – Visualisation du graphe d'appel . . . 10
3.11 C3 – Affinement ou analyse d'un autre projet . . . 11

4 Architecture et diagrammes **11**
4.1 Organisation des paquetages . . . 11
4.2 Modèle de faits . . . 12
4.3 Séquence d'une analyse . . . 12

5 Validation **13**
5.1 Comparaison avec les points de contrôle 13
5.2 Échantillons de robustesse 13

6 Difficultés et parties incomplètes **14**
6.1 Difficultés rencontrées 14
6.2 Parties incomplètes 15

7 Utilisation des outils d'intelligence artificielle **15**
7.1 Outil et périmètre d'utilisation 15
7.2 Exemples représentatifs 15
7.3 Vérification et décisions personnelles 15

8 Regard critique **15**
8.1 Limites de l'analyse statique 15
8.2 Pertinence des métriques 15
8.3 Améliorations possibles 16

9 Conclusion **16**

10 Références **16**

1 Objectifs et périmètre

1.1 Objectifs

L'objectif de ce TP est de développer un analyseur statique pour des projets Java à l'aide d'Eclipse JDT. À partir du dossier d'un projet, l'outil construit les arbres syntaxiques abstraits (AST), extrait les informations utiles et les place dans un modèle de faits indépendant de JDT.

Conformément au sujet, l'analyseur doit parcourir les AST, extraire la structure du projet et les appels de méthodes, calculer les 13 métriques demandées, puis construire un graphe d'appel interne. Il doit également être utilisable en ligne de commande grâce aux modes `structure`, `calls`, `metrics`, `graph` et `all`. Le paramètre X correspond au seuil utilisé pour rechercher les classes qui possèdent strictement plus de X méthodes.

1.2 Périmètre fonctionnel

L'entrée de l'outil est le chemin d'un projet Java. Tous les fichiers `.java` présents dans sa racine de sources sont analysés. Les informations extraites concernent principalement :

— les classes, interfaces, énumérations, records et types imbriqués ;

— les paquetages, attributs, méthodes, constructeurs et paramètres ;

— les relations d'héritage et d'implémentation ;

— les appels `MethodInvocation` et `SuperMethodInvocation`, avec le type statique du receveur et la méthode cible lorsque les bindings sont disponibles.

Le modèle de faits permet ensuite de distinguer les appels internes, externes et non résolus, de calculer les métriques de la partie B2 et de construire le graphe demandé dans la partie B3. Les résultats sont affichés dans la console selon le mode choisi par l'utilisateur.

1.3 Limites du périmètre

L'analyse reste statique : le programme étudié n'est pas exécuté. Une dépendance absente ou une erreur de syntaxe peut empêcher la résolution de certains bindings ; l'appel est alors indiqué comme non résolu, sans interrompre toute l'analyse. Les mécanismes dynamiques, comme la réflexion, ne sont pas traités spécifiquement.

2 Conception générale

2.1 Chaîne d'analyse

L'analyse est organisée comme une suite de traitements indépendants :

1. `ProjectSources` localise la racine des sources et recense les fichiers `.java` ;
2. `JdtParser` construit une unité de compilation pour chaque fichier et demande à JDT de résoudre les bindings ;
3. les visiteurs parcourent les AST et produisent le modèle de faits ;
4. `MetricsCalculator` calcule les métriques et `CallgraphBuilder` construit le graphe d'appel ;
5. `ConsoleReporter` affiche uniquement les résultats correspondant au mode demandé.

La classe `Main` coordonne ces étapes. Elle traite aussi les arguments de la ligne de commande et les erreurs, mais elle ne contient plus les calculs ni la mise en forme détaillée des résultats.

2.2 Architecture du projet

Le code est réparti en paquetages selon leur responsabilité :

— `cli` : lecture des arguments et choix du mode de sortie ;

— `parse` : recherche des sources et création des AST JDT ;

— `extract` : visiteurs chargés d'extraire la structure et les appels ;

— `model` : représentation des faits obtenus ;

— `metrics` : calcul des treize métriques ;

— `graph` : construction et interrogation du graphe d'appel ;

— `output` : affichage des résultats dans la console.

Cette séparation limite la dépendance envers Eclipse JDT aux étapes d'analyse et d'extraction. Les métriques, le graphe et l'affichage travaillent ensuite sur le modèle de faits, ce qui rend leur comportement plus simple à tester et à faire évoluer.

2.3 Représentation du modèle de faits

La figure 1 présente le modèle utilisé après le parcours des AST. `ProjectFact` regroupe les types et les fichiers sources ; un `TypeFact` possède des attributs et des méthodes, et chaque `MethodFact` conserve ses appels. Les identifiants de méthode prennent en compte les types des paramètres afin de distinguer les surcharges. Aucun objet du modèle ne dépend directement de JDT.

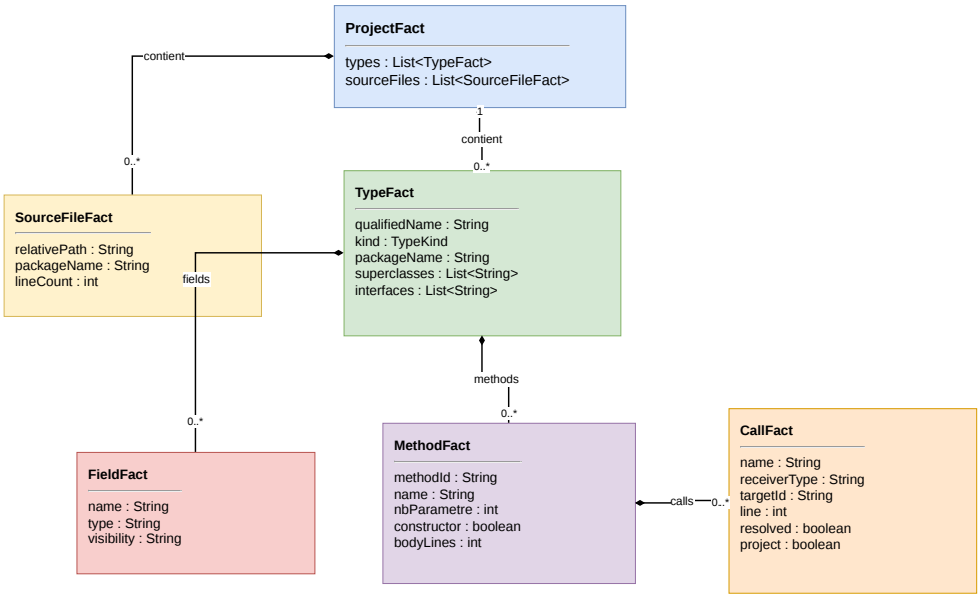


FIGURE 1 – Modèle de faits indépendant de JDT.

3 Réponses étape par étape

3.1 A0 – Environnement et projet de validation

3.1.1 Interprétation et stratégie

Cette étape vérifie que l'environnement est utilisable avant toute extraction. J'ai utilisé Java 17, Maven et Eclipse JDT Core 3.46.0. Le projet `resources/validation` a d'abord été compilé et exécuté seul, puis analysé par le projet `starter`.

3.1.2 Implémentation

`ProjectSources` localise la racine `src` et ordonne les fichiers Java. `JdtParser` crée ensuite une `CompilationUnit` par fichier. La configuration de `ASTParser` active la résolution des bindings et renseigne la racine des sources ainsi que le JDK avec `setEnvironment` ; ces informations sont indispensables pour résoudre les types entre plusieurs fichiers.

3.1.3 Résultats et validation

Les commandes `mvn clean package` puis `java -jar target/hai913i-tp1-analyzer.jar "../resources/validation" all 5` produisent le point de contrôle attendu : 14 unités de compilation et aucune erreur.

```
Racine des sources     : C:\Users\Administrator\Desktop\xue\M1S9\HAI913I - évolution et restructuration des logiciels\TP\TP1\resources\validation\src
Unites de compilation  : 14
Erreurs de compilation : 0
```

FIGURE 2 – Validation de l'environnement et du parseur JDT (A0).

3.2 A1 – Parcourir un AST

3.2.1 Interprétation et stratégie

Un visiteur dérivé de `ASTVisitor` affiche le type de chaque nœud. Un compteur de profondeur est incrémenté avant la visite des enfants puis décrémenté à la sortie du nœud ; il permet de restituer visuellement la hiérarchie de l'AST.

3.2.2 Implémentation

```java
@Override
public boolean preVisit2(ASTNode node) {
    String indentation = "  ".repeat(depth);

    System.out.println( indentation + node.getClass().getSimpleName() );

    depth++;

    return true;
}

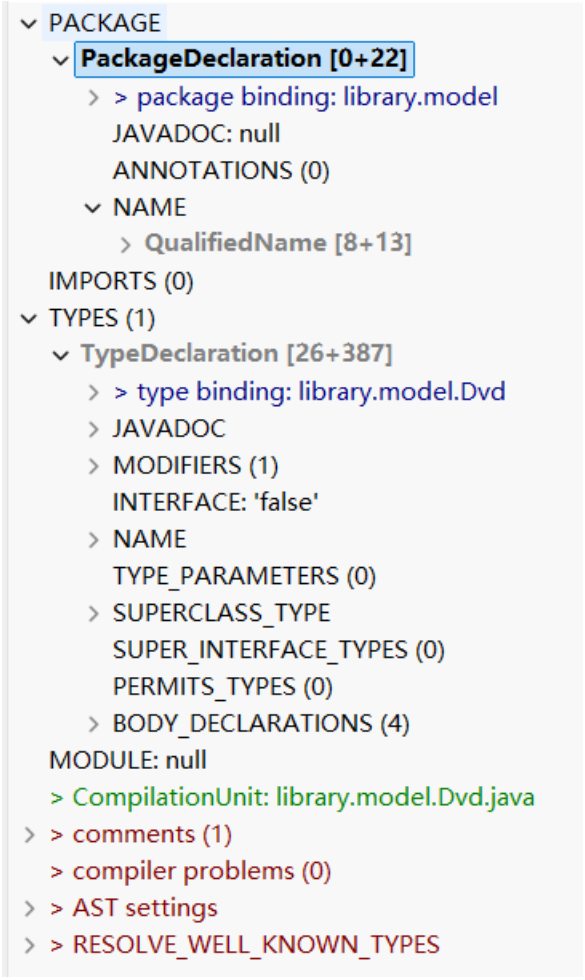
@Override
public void postVisit(ASTNode node) {
    depth--;
}
```

FIGURE 3 – Extrait ciblé du visiteur utilisé en A1

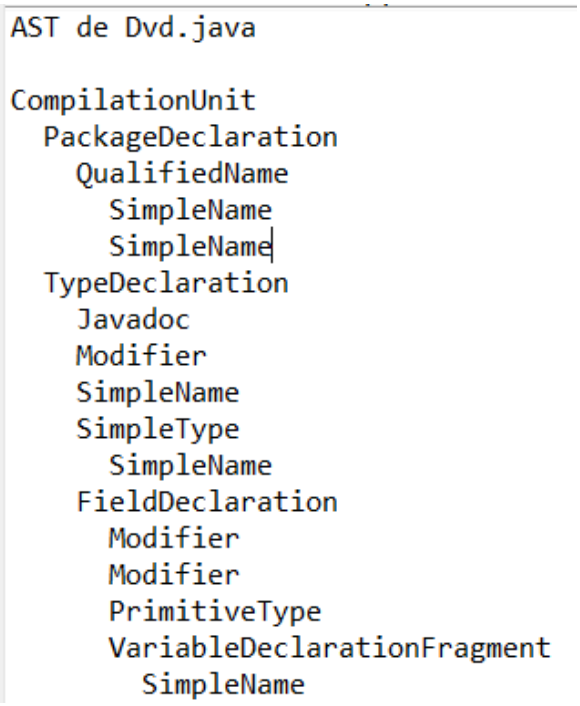
La valeur `true` autorise la visite des enfants du nœud. Avec `return false`, le nœud courant reste rencontré mais son sous-arbre est élagué ; les nœuds frères continuent d'être visités.

3.2.3 Résultats et validation

Pour `Dvd.java`, l'arbre contient un `TypeDeclaration`, un `FieldDeclaration` et trois `MethodDeclaration` ; le constructeur contient un `SuperConstructorInvocation`. Pour `Category.java`, le type est représenté par un `EnumDeclaration`, ce qui confirme qu'un visiteur limité à `TypeDeclaration` serait incomplet.



(a) `Dvd.java`



(b) `Category.java`

FIGURE 4 – Observation interactive des AST avec AST View (A1).

3.3 A2 – Extraire la structure du projet

3.3.1 Interprétation et stratégie

`StructureVisitor` construit un fait pour chaque classe, interface, énumération ou record, y compris les types imbriqués. Il extrait le nom qualifié, le paquetage, les superclasses, les interfaces directes, les attributs et les méthodes. Les constructeurs sont conservés comme des méthodes avec un indicateur dédié.

3.3.2 Implémentation

Les membres sont lus directement dans `bodyDeclarations()` du type courant. Cette stratégie évite d'attribuer à `LoanService` les membres de sa classe imbriquée `LoanService.Loan`. Les bindings fournissent les noms qualifiés et permettent de suivre la chaîne complète des superclasses.

```java
for (Object element : declaration.bodyDeclarations()) {
    BodyDeclaration bodyDeclaration = (BodyDeclaration) element;

    if (bodyDeclaration instanceof FieldDeclaration fieldDeclaration) {
        String baseType = fieldDeclaration.getType().toString();
        String visibility = visibilityOf(fieldDeclaration.getModifiers());

        for (Object fragmentObject : fieldDeclaration.fragments()) {
            VariableDeclarationFragment fragment = (VariableDeclarationFragment) fragmentObject;

            String declaredType = baseType + "[]".repeat(fragment.extraDimensions().size());

            fields.add(new FieldFact(
                    fragment.getName().getIdentifier(),
                    declaredType,
                    visibility
            ));
        }
    }
}
```

FIGURE 5 – Principe d'extraction des membres d'un type.

3.3.3 Résultats, validation et difficultés

Le résultat correspond exactement au point de contrôle : 15 types (12 classes, 2 interfaces et 1 énumération), 59 méthodes dont 9 constructeurs, et 21 attributs. En particulier, `LoanService` contient 6 méthodes et 5 attributs, tandis que `LoanService.Loan` contient 4 méthodes et 3 attributs. La chaîne de `LoanException` contient `java.lang.Exception` puis `java.lang.Throwable`. La principale difficulté a été de traiter uniformément les différentes sous-classes de `AbstractTypeDeclaration` et les bindings éventuellement absents, sans parcourir deux fois les types imbriqués.

```
Types        : 15
Classes      : 12
Interfaces   : 2
Enumerations : 1
Records      : 0
Fields       : 21
Methodes     : 59
Constructeurs : 9
```

FIGURE 6 – Synthèse de l'extraction structurelle sur le projet de validation (A2).

3.4 A3 – Extraire les appels et le type statique du receveur

3.4.1 Interprétation et stratégie

Un `CallVisitor` est appliqué au corps de chaque méthode. Il traite les `MethodInvocation` et les `SuperMethodInvocation`, y compris ceux placés dans une lambda. Chaque appel mémorise son nom, la ligne, le type statique du receveur, l'identifiant de la cible et son statut résolu, interne ou externe.

3.4.2 Implémentation

La cible est obtenue par `resolveMethodBinding()`, puis normalisée avec `getMethodDeclaration()` afin de retrouver la déclaration d'origine des méthodes génériques. Le type du receveur vient du binding de l'expression. Un binding nul n'interrompt pas l'analyse : l'appel est conservé avec la mention *non résolu*.

```
IMethodBinding binding = node.resolveMethodBinding();

String receiverType = findSuperReceiverType(node);
String targetMethod = findTargetMethod(binding);
int line =
        compilationUnit.getLineNumber(node.getStartPosition());

boolean resolved = binding != null;
boolean projectTarget = isProjectTarget(binding);
```

FIGURE 7 – Normalisation de la cible d'un appel.

3.4.3 Résultats, validation et difficultés

Sur le projet de validation, 94 appels sont extraits : 46 internes, 48 externes et aucun non résolu. Le cas `loanable.checkOut(member)` confirme que le receveur est bien `library.model.Loanable`. Les autres cas imposés sont également retrouvés : appel hérité de `Book` vers `Item#getTitle`, appel à `super.describe()` et appel dans la lambda de `Catalog.available`. Sur `robustness/missing-dep`, `StringUtils.capitalize` reste non résolu mais `Formatter.decorate` est résolu ; l'analyse se termine normalement.

```
Appels totaux          : 94
Appels vers le projet  : 46
Appels externes        : 48
Appels non resolus     : 0
```

(a) Comptage global

```
CALL checkOut
receiver=library.model.Loanable
target=library.model.Loanable#checkOut(library.model.Member)
line=36
resolved=true
project=true
```

(b) Type statique du receveur

FIGURE 8 – Validation de l'extraction des appels (A3).

3.5 B1 – Modèle de faits indépendant de JDT

3.5.1 Choix de représentation

Le modèle présenté en figure 1 est construit une seule fois à partir de toutes les unités de compilation. Les collections sont copiées en listes immuables et triées afin que deux exécutions produisent la même sortie. Un type imbriqué possède son propre nom qualifié, par exemple `library.service.LoanService.Loan`. Un appel non résolu est conservé explicitement au lieu d'être supprimé.

L'identifiant d'une méthode suit la forme `typeQualifie#nom(typesDesParametres)` ; un constructeur utilise `<init>`. Les types des paramètres distinguent les surcharges, par exemple les deux méthodes `Catalog#add`.

3.5.2 Indépendance vis-à-vis de JDT et validation

Les paquetages `metrics` et `graph` ne manipulent que `ProjectFact`, `TypeFact`, `MethodFact` et `CallFact` : aucune classe `org.eclipse.jdt` n'y apparaît. Les résultats A2 et A3 sont ainsi recalculés à partir du modèle sans relire le code source. Les appels entre fichiers, notamment de `LoanService` vers `Catalog`, restent résolus.

3.6 B2 – Calcul des métriques

3.6.1 Définitions et conventions

Les lignes de l'application correspondent aux lignes physiques des fichiers Java. Pour une méthode, seules les lignes comprises entre le début et la fin de son corps sont comptées ; les méthodes sans corps sont exclues de la moyenne. Pour un ensemble de taille n, la sélection des 10 % utilise $k = \lceil n/10 \rceil$ et conserve les égalités à la frontière. Le seuil X est strict : Q11 retient les classes possédant plus de X méthodes.

3.6.2 Résultats

Q	Mesure	Résultat
1–4	classes, lignes, méthodes, paquetages	15 ; 493 ; 59 ; 4
5–7	moyennes méthodes/classe, lignes/méthode, attributs/classe	3,93 ; 4,64 ; 1,40
8	10 % avec le plus de méthodes	Item (10), Member (6), LoanService (6)
9	10 % avec le plus d'attributs	Item (5), LoanService (5)
10	intersection de Q8 et Q9	Item, LoanService
11	plus de 5 méthodes	Item, Member, LoanService
12	exemples du maximum par classe	Item : checkOut (7) ; LoanService : borrow (14)
13	nombre maximal de paramètres	5, Statistics#report

TABLE 1 – Métriques obtenues sur le projet de validation.

Les 4,64 lignes de Q6 correspondent à 246 lignes réparties sur 53 méthodes ayant un corps. Pour Q12, les égalités sont conservées : les six méthodes de `Member` ont chacune trois lignes, tandis que `Loanable` ne possède aucune méthode avec corps. Ces résultats sont identiques aux valeurs de référence du sujet.

```
Metriques B2
------------
Q1  Nombre de classes              : 15
Q2  Lignes de l'application        : 493
Q3  Nombre total de methodes       : 59
Q4  Nombre total de paquetages     : 4
Q5  Moyenne methodes / classe      : 3.93
Q6  Moyenne lignes / methode       : 4.64
Q7  Moyenne attributs / classe     : 1.40
```

(a) Métriques Q1 à Q7

```
Q8  10 % des classes avec le plus de methodes :
    library.model.Item : 10
    library.model.Member : 6
    library.service.LoanService : 6
Q9  10 % des classes avec le plus d'attributs :
    library.model.Item : 5
    library.service.LoanService : 5
Q10 Intersection de Q8 et Q9 :
    library.model.Item
    library.service.LoanService
Q11 Classes avec strictement plus de 5 methodes :
    library.model.Item : 10
    library.model.Member : 6
    library.service.LoanService : 6
```

(b) Classements Q8 à Q11

FIGURE 9 – Extraits de la sortie console des métriques B2.

3.7 B3 – Graphe d'appel

3.7.1 Construction du graphe

Chaque méthode du projet devient un nœud. Un appel résolu vers une autre méthode du projet crée un arc orienté de l'appelante vers l'appelée ; plusieurs sites entre les mêmes méthodes sont agrégés dans le poids de l'arc. Les appels externes et non résolus sont comptés mais ne créent pas de nœud. Deux index, sortant et entrant, permettent d'obtenir respectivement les appelées et les appelantes. La sortie textuelle a été retenue car elle est lisible en console, facile à comparer et ne nécessite aucune bibliothèque supplémentaire.

3.7.2 Résultats et validation

Le graphe contient 59 nœuds et 44 arcs, pour 46 sites internes, 48 appels externes et 0 appel non résolu. L'arc `Catalog#add(Item,Item) -> Catalog#add(Item)` a un poids de 2 et `TextUtils#repeat` possède une boucle récursive. La requête inverse sur `Loanable#checkOut(Member)` retourne uniquement `LoanService#borrow` ; `Item#checkOut(Member)` n'a aucune appelante, conformément à l'absence d'expansion CHA.

```
Graphe d'appel B3
-----------------
Noeuds              : 59
Arcs                : 44
Sites internes      : 46
Appels externes     : 48
Appels non resolus  : 0
```

FIGURE 10 – Synthèse du graphe d'appel obtenu sur le projet de validation.

3.8 B4 – Outil utilisable et reproductible

3.8.1 Interface en ligne de commande

L'outil s'exécute avec la syntaxe suivante :

```
java -jar target/hai913i-tp1-analyzer.jar DOSSIER MODE X
```

`MODE` vaut `structure`, `calls`, `metrics`, `graph` ou `all`. Le seuil X est utilisé par Q11. La classe `Main` orchestre l'analyse, tandis que `CommandLineOptions`, `ConsoleReporter`, `MetricsCalculator` et `CallGraphBuilder` assument des responsabilités séparées.

3.8.2 Gestion des erreurs et reproductibilité

Cas	Code	Comportement
arguments ou X invalides	2	message d'usage et refus explicite
chemin inexistant	3	message indiquant que le chemin n'est pas un répertoire
aucun fichier Java	4	message explicite, analyse impossible
erreur inattendue	10	message court, sans pile d'exception brute

TABLE 2 – Principaux codes de sortie de l'analyseur.

Les erreurs de syntaxe et les dépendances manquantes sont signalées sans arrêter l'extraction des fichiers encore exploitables. Les échantillons `syntax-error` et `missing-dep` terminent donc avec le code 0 après avoir affiché leurs diagnostics ; un chemin inexistant et `X=abc` produisent respectivement les codes 3 et 2. Enfin, `mvn clean package` suivi des commandes documentées dans le README reproduit les points de contrôle sur une copie fraîche, y compris lorsque le chemin contient des espaces.

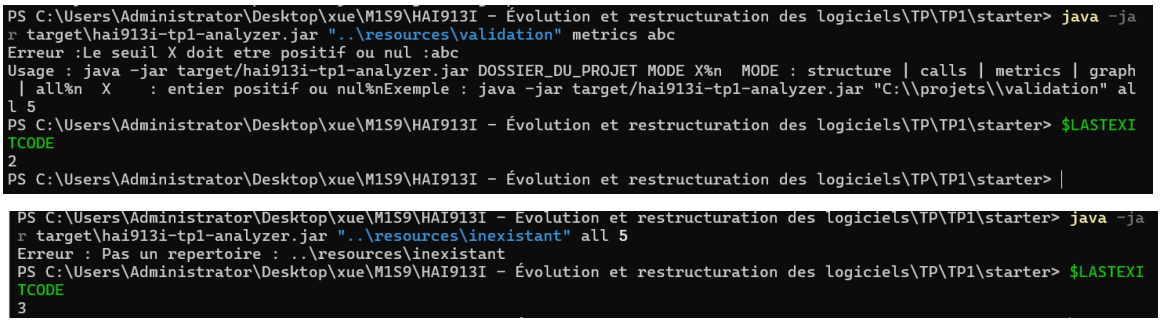


FIGURE 11 – Refus d'un seuil invalide et d'un chemin inexistant, avec codes de sortie 2 et 3.

3.9 C1 – Extraction avec Spoon et comparaison

3.9.1 Stratégie

L'extraction a été réimplémentée avec Spoon 11.5.0 sans modifier la chaîne JDT des parties obligatoires. Le `Launcher` construit un `CtModel`, puis les éléments Spoon sont convertis vers les mêmes `TypeFact`, `FieldFact`, `MethodFact` et `CallFact` que ceux produits par JDT. Les calculateurs de métriques et le constructeur du graphe restent donc indépendants de la bibliothèque d'analyse employée. Une option `noClasspath` permet également de construire le modèle lorsqu'une dépendance externe est absente.

3.9.2 Différences de comptage

Le tableau 3 distingue les valeurs brutes de Spoon des valeurs obtenues après application des conventions du TP.

Mesure	JDT	Spoon brut	Spoon normalisé
Types	15	15	15
Attributs	21	24	21
Méthodes et constructeurs	59	63	59
Appels	94	107	94

TABLE 3 – Comparaison des comptages JDT et Spoon sur le projet de validation.

Trois différences principales expliquent ces écarts. Premièrement, `getAllTypes()` ne renvoie que les 14 types de premier niveau ; une requête sur tous les `CtType` retrouve aussi la classe interne `LoanService.Loan`, soit 15 types. Deuxièmement, Spoon représente les trois constantes de l'énumération `Category` comme des `CtField`. Leur exclusion par `CtEnumValue` ramène le total de 24 à 21 attributs. Troisièmement, Spoon sépare 50 `CtMethod` et 13 `CtConstructor`, dont quatre implicites. L'exclusion de ces derniers redonne les 59 exécutables du TP. Enfin, les 107 `CtInvocation` comprennent 13 appels de constructeur à `super(...)` ; leur exclusion redonne les 94 appels comptés avec JDT.

Après normalisation, les deux extracteurs produisent le même graphe : 59 nœuds, 44 arcs et 46 sites d'appel internes. La figure 12 montre cette comparaison exécutée sur le projet de validation.

```
Mode Spoon noClasspath : false

Comparaison JDT / Spoon
-----------------------
Mesure                      JDT   Spoon brut  Spoon normalise
Types                        15           15               15
Attributs                    21           24               21
Methodes                     59           63               59
Appels                       94          107               94

Types Spoon premier niveau : 14

Graphe JDT   : 59 noeuds, 44 arcs, 46 sites internes
Graphe Spoon : 59 noeuds, 44 arcs, 46 sites internes

Appels Spoon non resolus :
  aucun
```

FIGURE 12 – Comparaison des résultats JDT et Spoon après normalisation.

3.9.3 Robustesse en mode `noClasspath`

Sur `resources/robustness/missing-dep`, Spoon construit encore un modèle et peut déduire le type `org.apache.commons.lang3.StringUtils` depuis l'import. Cette information devinée n'est cependant pas considérée comme une résolution : en l'absence de déclaration exécutable, l'appel `capitalize` reste marqué non résolu. L'appel interne vers `Formatter.decorate` reste quant à lui résolu. Ce comportement est illustré par la figure 13.

```
Mode Spoon noClasspath : true

Comparaison JDT / Spoon
-----------------------
Mesure                      JDT   Spoon brut  Spoon normalise
Types                         2            2                2
Attributs                     1            1                1
Methodes                      2            4                2
Appels                        2            4                2

Types Spoon premier niveau : 2

Graphe JDT   : 2 noeuds, 1 arcs, 1 sites internes
Graphe Spoon : 2 noeuds, 1 arcs, 1 sites internes

Appels Spoon non resolus :
  aucun
```

FIGURE 13 – Analyse du projet `missing-dep` avec Spoon en mode `noClasspath`.

3.10 C2 – Visualisation du graphe d'appel

Le graphe B3 est exporté au format DOT par `DotExporter`. L'exporteur parcourt les mêmes nœuds et arcs que `ConsoleReporter`, les trie pour garantir une sortie déterministe, puis échappe les identifiants utilisés comme libellés. Graphviz transforme ensuite le fichier DOT en PNG ou en PDF ; aucune nouvelle analyse du code source n'est effectuée.

L'export complet contient les 59 nœuds et 44 arcs du projet de validation. Comme cette représentation est dense, une option de filtrage conserve uniquement les méthodes d'un paquetage

et les arcs dont les deux extrémités appartiennent à ce paquetage. Le filtre `library.service` fournit ainsi une vue plus lisible, adaptée au rapport, tout en restant une projection du même `CallGraph`.



FIGURE 14 – Graphe d'appel filtré sur le paquetage `library.service`.

Les commandes de génération sont documentées dans le README afin de reproduire le résultat depuis une copie fraîche du dépôt. Le format DOT reste exploitable sans interface graphique et le PDF conserve des libellés lisibles lors d'un agrandissement.

3.11 C3 – Affinement ou analyse d'un autre projet

Cette extension optionnelle n'a pas été réalisée. Le travail bonus a été limité à la comparaison JDT/Spoon et à la visualisation du graphe, sans modifier les résultats validés des parties A et B.

4 Architecture et diagrammes

4.1 Organisation des paquetages

La figure 15 reprend les dépendances importantes du code final. `parse` et `extract` constituent la frontière avec JDT. Une fois le `ProjectFact` construit, les composants de métriques et de graphe ne reviennent jamais aux AST. `output` ne calcule aucune donnée : il se limite à leur présentation.

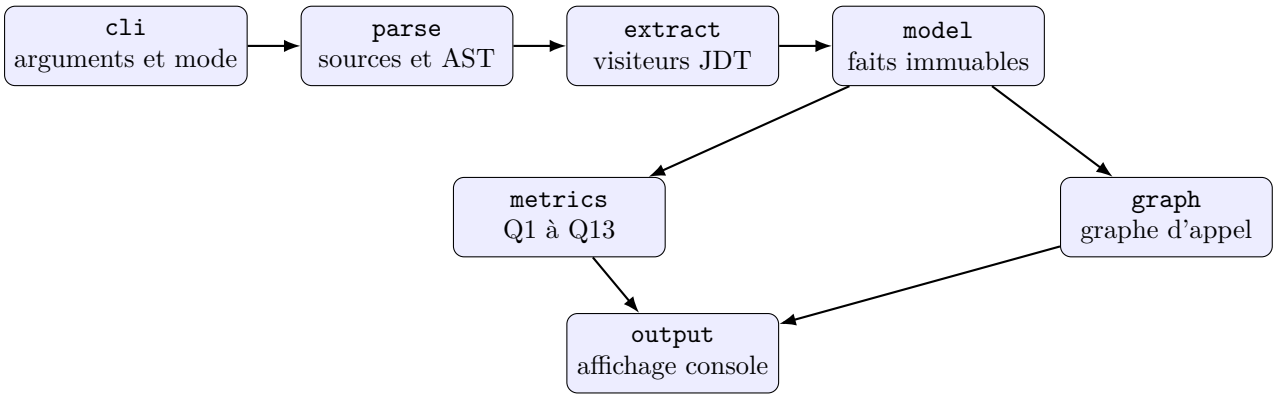


FIGURE 15 – Organisation des composants et circulation des données.

4.2 Modèle de faits

Le diagramme de la figure 1, présenté dans la conception générale, constitue le diagramme de classes utile du projet. Il montre que `ProjectFact` agrège les fichiers sources et les types, qu'un type possède ses attributs et méthodes, et qu'une méthode conserve ses appels. Les dépendances JDT s'arrêtent avant ce modèle, ce qui permet de calculer B2 et B3 sans réanalyser les sources.

4.3 Séquence d'une analyse

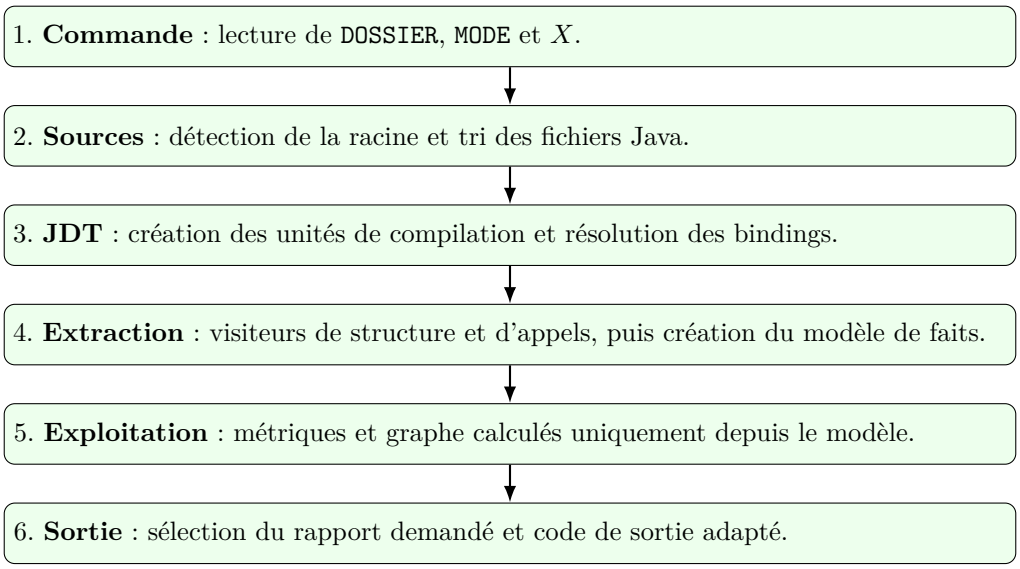


FIGURE 16 – Séquence d'une exécution complète de l'analyseur.

5 Validation

5.1 Comparaison avec les points de contrôle

Étape	Mesure	Attendu	Obtenu	État
A0	Unités de compilation	14	14	Conforme
A0	Erreurs de compilation	0	0	Conforme
A2	Types	15	15	Conforme
A2	Méthodes	59	59	Conforme
A2	Constructeurs	9	9	Conforme
A2	Attributs	21	21	Conforme
A3	Appels totaux	94	94	Conforme
A3	Internes / externes / non résolus	46 / 48 / 0	46 / 48 / 0	Conforme
B2	Q1–Q7	réf. sujet	mêmes valeurs	Conforme
B3	Nœuds / arcs / sites internes	59 / 44 / 46	59 / 44 / 46	Conforme

TABLE 4 – Comparaison avec les principaux points de contrôle du sujet.

Les classements de B2 ont aussi été comparés manuellement : pour $X = 5$, Q11 retourne `Item`, `Member` et `LoanService` ; Q13 retourne `Statistics#report` avec cinq paramètres. Pour B3, le poids 2 de l'arc entre les deux surcharges de `Catalog#add`, la récursion de `TextUtils#repeat` et la requête inverse sur `Loanable#checkOut` ont été vérifiés séparément.

5.2 Échantillons de robustesse

Échantillon	Comportement observé	Résultat
`missing-dep`	Deux erreurs JDT sont affichées. `capitalize` reste non résolu, mais `Formatter.decorate` est résolu et le graphe est construit.	code 0
`syntax-error`	Les deux erreurs de `Broken.java` sont signalées ; `Broken` et `Helper` restent analysés.	code 0
`no-java-files`	L'absence de fichier Java produit un message explicite et interrompt proprement l'analyse.	code 4
chemin inexistant	Le chemin est refusé avant le parsing.	code 3
$X =$ `abc`	Le seuil est refusé et le message d'usage est affiché.	code 2

TABLE 5 – Validation des cas dégradés et des entrées invalides.

Sur `missing-dep`, le modèle contient encore deux types et deux méthodes. Les deux sites d'appel sont conservés : un appel interne forme l'unique arc du graphe et l'autre est compté comme non résolu. Ce résultat montre qu'un binding absent n'est pas traité comme une erreur fatale.

```
CALL capitalize
receiver=non resolu
target=non resolu
line=14
resolved=false
project=false

CALL decorate
receiver=demo.Formatter
target=demo.Formatter#decorate(java.lang.String)
line=15
resolved=true
project=true


Types         : 2
Classes       : 2
Interfaces    : 0
Enumerations  : 0
Records       : 0
Fields        : 1
Methodes      : 2
Constructeurs : 0

Appels totaux           : 2
Appels vers le projet   : 1
Appels externes         : 0
Appels non resolus      : 1
```

FIGURE 17 – Appel résolu et appel non résolu avec une dépendance manquante.

6 Difficultés et parties incomplètes

6.1 Difficultés rencontrées

La première difficulté concernait la hiérarchie des nœuds JDT : une interface est représentée par `TypeDeclaration`, mais une énumération par `EnumDeclaration`. Le traitement a donc été factorisé autour de `AbstractTypeDeclaration`, tout en gardant les opérations spécifiques à chaque sous-type.

La seconde difficulté était le risque de double comptage et de mauvaise attribution des membres. Une visite récursive globale attribuait les membres de `LoanService.Loan` au type englobant. La lecture de `bodyDeclarations()` et la création séparée d'un fait par type ont supprimé ce problème.

Enfin, la résolution des appels ne peut jamais supposer qu'un binding existe. Les appels non résolus sont donc représentés dans `CallFact`, puis comptés séparément. Cette décision permet de continuer sur les projets incomplets et a été validée avec `missing-dep`. La dernière étape a consisté à sortir les calculs et l'affichage de `Main` afin de rendre la ligne de commande plus lisible et chaque composant testable isolément.

6.2 Parties incomplètes

Aucune partie obligatoire de A0 à B4 n'est laissée incomplète. Le graphe suit volontairement la sémantique principale du sujet : il ne réalise pas d'expansion CHA et ne crée pas d'arcs vers les constructeurs. Les appels réfléchis ou produits dynamiquement ne sont pas reconstruits. La partie C, indiquée comme bonus optionnel, n'a pas été implémentée et n'affecte pas les résultats obligatoires.

7 Utilisation des outils d'intelligence artificielle

7.1 Outil et périmètre d'utilisation

Nous avons utilisé ChatGPT/Codex, basé sur le modèle GPT-5, comme assistant pendant le TP. L'outil a servi à reformuler certaines consignes, expliquer les API de JDT, proposer un découpage incrémental, interpréter des messages d'erreur Java et structurer le README ainsi que le présent compte-rendu. Il n'a pas remplacé l'exécution du programme ni la comparaison avec les points de contrôle.

7.2 Exemples représentatifs

Trois usages sont représentatifs. Premièrement, l'IA a aidé à distinguer `TypeDeclaration`, `EnumDeclaration` et `AbstractTypeDeclaration` après une erreur de surcharge de `visit`. Deuxièmement, elle a expliqué pourquoi un parcours récursif pouvait doubler les types et attribuer les champs d'une classe imbriquée à la classe externe. Troisièmement, elle a proposé de séparer l'affichage dans `ConsoleReporter` et les arguments dans `CommandLineOptions` lors de la refactorisation B4.

7.3 Vérification et décisions personnelles

Chaque proposition a été vérifiée par compilation Maven, exécution sur le projet de validation et comparaison avec les valeurs du sujet. Les cas `missing-dep`, `syntax-error`, le seuil invalide et le chemin inexistant ont servi à vérifier les conseils concernant la robustesse. Les choix définitifs du modèle de faits, des identifiants de méthodes, des conventions de comptage, de la sémantique du graphe et de l'interface de commande ont été relus et adaptés au code réellement développé. Aucun historique complet de conversation n'est joint au rapport.

8 Regard critique

8.1 Limites de l'analyse statique

Le graphe représente les cibles déterminées par les bindings JDT, mais il ne prédit pas toutes les cibles possibles d'un appel virtuel. Dans l'appel à `Loanable#checkOut`, le receveur reste donc l'interface déclarée ; aucune expansion CHA ou RTA n'ajoute automatiquement les implémentations. La réflexion, les proxys, le chargement dynamique, l'injection de dépendances et le code généré peuvent également cacher des appels. Enfin, un classpath incomplet dégrade la résolution des bindings, même si l'outil conserve les appels concernés comme non résolus.

8.2 Pertinence des métriques

Les métriques indiquent la taille et certaines concentrations structurelles. Elles permettent par exemple d'identifier `Item` et `LoanService` comme des classes à examiner en priorité, car elles cumulent de nombreux attributs et méthodes. Elles ne démontrent toutefois ni un défaut ni une mauvaise conception : une classe centrale peut légitimement avoir des valeurs élevées. Les lignes de code dépendent aussi du formatage, tandis que le nombre de paramètres ou de

méthodes ignore leur complexité sémantique. Ces résultats doivent donc guider une inspection, et non remplacer un jugement architectural.

8.3 Améliorations possibles

Les améliorations prioritaires seraient l'ajout de tests unitaires pour chaque calcul de B2 et pour les requêtes du graphe, puis un export DOT ou JSON facilitant l'exploitation automatique. Une résolution interprocédurale fondée sur CHA ou RTA améliorerait les appels virtuels, au prix de plus de nœuds et d'arcs potentiels. La prise en charge de projets Maven multi-modules et de leur classpath réel réduirait les bindings non résolus. Enfin, des diagnostics regroupés par fichier rendraient l'analyse de grands projets plus lisible.

9 Conclusion

Ce TP a conduit à un analyseur Java complet fondé sur Eclipse JDT. Il transforme plusieurs unités de compilation en un modèle de faits indépendant, puis calcule les treize métriques demandées et un graphe d'appel interrogeable dans les deux sens. Les points de contrôle A0 à B4 sont retrouvés sur le projet de validation, et les échantillons dégradés montrent que les erreurs de syntaxe ou les bindings absents n'entraînent pas d'arrêt brutal.

Le principal apprentissage est que la fiabilité des métriques et du graphe dépend d'abord de la qualité des faits extraits : gestion des types imbriqués, identifiants stables, types statiques et bindings nuls. La séparation entre parsing, extraction, modèle, calculs et affichage rend la solution reproductible et prépare les extensions proposées, notamment l'export de graphe et une résolution plus précise des appels virtuels.

10 Références

Références

[1] Bachar Rima, *HAI913I – TP 1 : Analyse statique*, Université de Montpellier, année 2026–2027.

[2] Bachar Rima, *HAI913I – CM 02 : Analyse statique des programmes*, Université de Montpellier, année 2026–2027.

[3] Bachar Rima, *HAI913I – TP 1, annexe Eclipse JDT et Spoon*, Université de Montpellier, année 2026–2027.

[4] Bachar Rima, *HAI913I – TP 1, annexe Visitor*, Université de Montpellier, année 2026–2027.

[5] Eclipse Foundation, *Javadoc de ASTParser*, https://help.eclipse.org/latest/topic/org.eclipse.jdt.doc.isv/reference/api/org/eclipse/jdt/core/dom/ASTParser.html, consulté le 11 octobre 2026.

[6] Eclipse Foundation, *Javadoc de ASTVisitor*, https://help.eclipse.org/latest/topic/org.eclipse.jdt.doc.isv/reference/api/org/eclipse/jdt/core/dom/ASTVisitor.html, consulté le 11 octobre 2026.